A Joint Communication and Learning Framework for Hierarchical Split Federated Learning

Latif U. Khan [1], Mohsen Guizani [2], Ala Al-Fuqaha [2], Choong Seon Hong [2], Dusit Niyato [2], and Zhu Han [2]

[1]Mohamed Bin Zayed University of Artificial Intelligence
[2]Affiliation not available

August 31, 2023

Abstract

In contrast to methods relying on a centralized training, emerging Internet of Things (IoT) applications can employ federated learning (FL) to train a variety of models for performance improvement and improved privacy preservation. FL calls for the distributed training of local models at end-devices, which uses a lot of processing power (i.e., CPU cycles/sec). Most end-devices have computing power limitations, such as IoT temperature sensors. One solution for this problem is split FL. However, split FL has its problems including a single point of failure, issues with fairness, and a poor convergence rate. We provide a novel framework, called hierarchical split FL (HSFL), to overcome these issues. On grouping, our HSFL framework is built. Partial models are constructed within each group at the devices, with the remaining work done at the edge servers. Each group then performs local aggregation at the edge following the computation of local models. End devices are given access to such an edge aggregated model so they can update their models. For each group, a unique edge aggregated HSFL model is produced by this procedure after a set number of rounds. Shared among edge servers, these edge aggregated HSFL models are then aggregated to produce a global model. Additionally, we propose an optimization problem that takes into account the RLA of devices, transmission latency, transmission energy, and edge servers' compute latency in order to reduce the cost of HSFL. The formulated problem is a mixed-integer non-linear programming (MINLP) problem and cannot be solved easily. To tackle this challenge, we perform decomposition of the formulated problem to yield sub-problems. These sub-problems are edge computing resource allocation problem and joint relative local accuracy (RLA) minimization, wireless resource allocation, task offloading, and transmit power allocation sub-problem. Due to the convex nature of edge computing, resource allocation is done so utilizing a convex optimizer, as opposed to a block successive upper-bound minimization (BSUM) based approach for joint relative local accuracy (RLA) minimization, resource allocation, job offloading, and transmit power allocation. Finally, we present the performance evaluation findings for the proposed HSFL scheme.

A Joint Communication and Learning Framework for Hierarchical Split Federated Learning

Latif U. Khan, Mohsen Guizani, *Fellow, IEEE*, Ala Al-Fuqaha, *Senior Member, IEEE*, Choong Seon Hong, *Senior Member, IEEE*, Dusit Niyato, *Fellow, IEEE*, Zhu Han, *Fellow, IEEE*

***Abstract*—In contrast to methods relying on a centralized training, emerging Internet of Things (IoT) applications can employ federated learning (FL) to train a variety of models for performance improvement and improved privacy preservation. FL calls for the distributed training of local models at end-devices, which uses a lot of processing power (i.e., CPU cycles/sec). Most end-devices have computing power limitations, such as IoT temperature sensors. One solution for this problem is split FL. However, split FL has its problems including a single point of failure, issues with fairness, and a poor convergence rate. We provide a novel framework, called hierarchical split FL (HSFL), to overcome these issues. On grouping, our HSFL framework is built. Partial models are constructed within each group at the devices, with the remaining work done at the edge servers. Each group then performs local aggregation at the edge following the computation of local models. End devices are given access to such an edge aggregated model so they can update their models. For each group, a unique edge aggregated HSFL model is produced by this procedure after a set number of rounds. Shared among edge servers, these edge aggregated HSFL models are then aggregated to produce a global model. Additionally, we propose an optimization problem that takes into account the RLA of devices, transmission latency, transmission energy, and edge servers' compute latency in order to reduce the cost of HSFL. The formulated problem is a mixed-integer non-linear programming (MINLP) problem and cannot be solved easily. To tackle this challenge, we perform decomposition of the formulated problem to yield sub-problems. These sub-problems are edge computing resource allocation problem and joint relative local accuracy (RLA) minimization, wireless resource allocation, task offloading, and transmit power allocation sub-problem. Due to the convex nature of edge computing, resource allocation is done so utilizing a convex optimizer, as opposed to a block successive upper-bound minimization (BSUM) based approach for joint relative local accuracy (RLA) minimization, resource allocation, job offloading, and transmit power allocation. Finally, we present the performance evaluation findings for the proposed HSFL scheme.**

***Index Terms*—Federated learning, Internet of Things, split learning, hierarchical federated learning.**

L. U. Khan is with the Machine Learning Department, Mohamed Bin Zayed University of Artificial Intelligence (MBZUAI), Abu Dhabi, United Arab Emirates and also wiht the Department of Computer Science & Engineering, Kyung Hee University, Yongin-si 17104, South Korea. [Email: latif.u.khan2@gmail.com, latif.khan@mbzuai.ac.ae]

M. Guizani is with the Machine Learning Department, Mohamed Bin Zayed University of Artificial Intelligence (MBZUAI), Abu Dhabi, United Arab Emirates. [Email: mguizani@ieee.org]

A. Al-Fuqaha is with Computer Science Department, College of Engineering Applied Sciences, Hamad Bin Khalifa University, Qatar. [Email: aalfuqaha@hbku.edu.qa]

C. S. Hong is with the Department of Computer Science & Engineering, Kyung Hee University, Yongin-si 17104, South Korea. [Email: cshong@khu.ac.kr]

D. Niyato is with the School of Computer Science and Engineering, Nanyang Technological University, Singapore.[Email: DNIYATO@ntu.edu.sg]

Zhu Han is with the Electrical and Computer Engineering Department, University of Houston, Houston, TX 77004 USA, and also with the Computer Science Department, University of Houston, Houston, TX 77004 USA, and the Department of Computer Science and Engineering, Kyung Hee University, South Korea. [Email: hanzhu22@gmail.com]

Corresponding authors: Latif U. Khan and C. S. Hong.

I. Introduction

Applications for the Internet of Things (IoT) are designed to serve a large number of users by satisfying their varied needs [1]–[6]. IoT solutions must be carefully designed in order to meet these needs. Effectively modeling of IoT network functions can help us optimize the performance of IoT systems. These techniques can be based on a variety of theories, including optimization theory, game theory, graph theory, and heuristics. However, some IoT issues could seem challenging to accurately model using the aforementioned techniques [7]–[10]. One can utilize machine learning (ML) to get around this restriction. Since centralized machine learning relies on moving data from end devices to a single location for training, and thus it suffers from privacy leaks. Federated learning (FL), which does not need moving data from devices to a centralized server for training, can be used to address the privacy leakage problem associated with centralized ML [11]–[13]. Devices in FL learn their local models and send them to a server at the edge or in the cloud where they are aggregated to produce a global model. FL has some challenges, including resource optimization, single point of failure (SPF), incentive design, and learning algorithm design, despite the fact that it can maintain privacy more effectively than centralized ML [14]–[16]. Additionally, a local learning model can also be unable to be trained within the allotted time on devices with insufficient computational power.

One aggregation server in FL has the potential to be attacked by an unauthorized user or malfunction as a result of physical damage, which would degrade FL performance [11]. To motivate the end-devices towards learning a global FL model, the works in [17] and [18] presented the incentive mechanisms based on Stackelberg game and auction theory, respectively. The works in [19]–[21] presented the concept of split FL to resolve the issues of resource constraints in IoT devices. Different from the existing works [19]–[21], we propose a novel hierarchical split FL (HSFL) framework. We consider resource optimization, resolve the SPF issue, and perform hierarchical aggregations to improve the convergence

speed for resource-constrained devices. The issue of SPF can be resolved by enabling FL with distributed aggregation so as to avoid a CPF. Also, we modify the learning algorithm by performing local aggregations prior to global aggregation to improve the convergence speed. The issue of resource-constrained devices can be resolved by performing partial local model learning at devices and remaining at the edge servers in a similar fashion as that of traditional split FL. Note that our proposed framework is based on distributed aggregations of edge aggregated models and is based on true decentralization. In traditional hierarchical FL [22]–[24], a leader and follower relationship is followed. A global aggregation aggregator acts as a leader and devices as followers. In this fashion of traditional hierarchical FL, some of the groups used for edge aggregated models may perform better compared to others and thus will influence the global model more. In our proposed decentralized aggregation-based HSFL, different groups can be tuned for edge aggregations and local iterations based on their performance to obtain a fairness-enabled global FL model. The key contributions of the article are given below.

- We propose a novel framework, namely, HSFL. The latter computes partial local models at end-devices and the remaining at the edge server. Additionally, distribution aggregations are considered to avoid the SPF issue. To improve the convergence speed, HSFL uses local aggregations prior to global aggregation. The HSFL provides a tradeoff between communication cost due to edge aggregations and global HSFL accuracy. An increase in edge aggregations will improve its convergence but at the cost of communication cost due to edge aggregations.
- A cost function that accounts for overall latency (i.e., relative local accuracy (RLA), energy consumption (i.e., transmission energy), edge server computing delay), and transmission delay is defined in this work.
- Since the formulated problem is non-convex, we divide the main problem into two sub-problems: joint RLA minimization, transmit power allocation, resource allocation, and association sub-problem, and edge servers computing resource optimization. We employ a convex optimization-based method for allocating computing resources to edge servers. We utilize a block successive upper-bound minimization (BSUM) based technique for the joint RLA minimization, transmit power allocation, resource allocation, and association sub-problem due to its non-convex nature.
- We conclude by presenting the findings from our performance assessment of the proposed HSFL framework.

II. Related Works

A. Resource Optimization in FL

Various works [25]–[29] discussed resource optimization for FL. The work in [25] took into account FL over wireless networks and developed a problem to reduce the overall FL time. For optimizing the wireless resource allocation and local device operating frequencies, the authors proposed a heuristic approach for reducing the global FL time. Despite the fact that the authors have demonstrated a performance gain, the high complexity of the heuristic strategy may not be preferred. Additionally, the performance of [25] can be further improved by performing efficient power allocation and association in the case of multiple base stations. The work in [26] discussed FL incentive systems and resource optimization. They also presented open challenges and a Stackelberg game-based incentive mechanism. Another work [30] proposed a framework for collaborative learning and communication for FL across wireless networks. The authors discussed how FL performance is impacted by packet error rate. They formulated an optimization problem to reduce the FL cost by jointly optimizing resource and transmit power allocation. They employed bipartite matching for resource allocation while deriving an equation for optimal power allocation. Analysis of complexity was also carried out. Wang *et al.* in [27] offered a control strategy for resource-constrained scenarios that provided a trade-off between local model update and global model aggregation for the loss function minimization. Finally, the authors used a real-time dataset to validate their idea. The authors in [28] analyzed wireless FL. They formulated an optimization problem and offered two types of tradeoffs: (a) communication and computing latencies and (b) FL model computing time and local energy consumption. All of the works in [25]–[28] are using a single aggregation server which has the downside of failure in case of a centralized server failure.

B. Split FL

The works in [19]–[21] considered split learning. The authors in [19] presented a split neural network for collaborative healthcare institutions. In split learning, a partial model is learned at devices, and the rest of the layers are computed at the server. They also identified two cases of split neural networks. Finally, the authors presented simulation results for a split neural network using CIFAR-100 and CIFAR-10 datasets. In another work [20], split learning and FL were coupled for devices with limited resources by the authors. In their framework, a set of resource-constrained devices learn a part of local learning models and send the remaining to nearby servers for computation. At the central server, the local models are combined to produce a global model after the whole local learning models have been computed through interaction between the servers and the devices. The end devices then receive this global model to further update their local model. Finally, the authors validated their concept using CIFAR-10, MNIST, and FMNIST. The authors in [21] considered split learning and federated learning. Their framework consists of a device set which computes a partial local model. Then, the partial local model is shared with their corresponding servers. Additionally, they used a dataset of chest X-rays for COVID-19 recognition and MNIST to assess how well their suggested architecture performed. All of the works in [19]–[21] presented split learning, but they did not take into account problems with robustness brought on by a single point of failure. Additionally, the proposals of [19]–[21] must be improved to get faster convergence. Also, there must be an effective analysis of split learning over wireless networks.

TABLE I: Comparison of related works with proposed work.

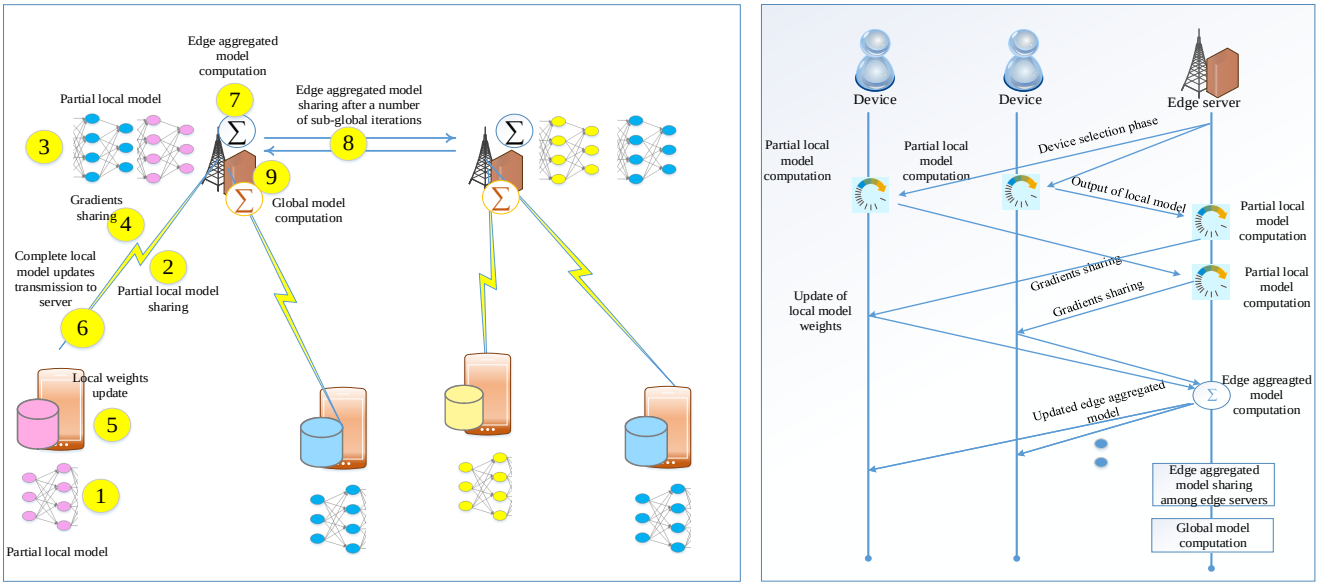
Reference	Resource optimization	Offloading	Robustness	HFL	SFL	Remark
Khan *et al.*, [25]	✓	✗	✗	✓	✗	This work proposes a novel dispersed FL framework for autonomous cars.
Khan *et al.*, [26]	✓	✗	✗	✗	✗	This reviewed resource optimization and incentive mechanism. Moreover, they presented a case study for incentive mechanism design.
Wang *et al.*, [27]	✓	✗	✗	✗	✗	This work offered a control strategy for resource-constrained scenarios that provided a trade-off between global model aggregation and local model update in order to minimize the loss function.
Dinh *et al.*, [28]	✓	✗	✗	✗	✗	This work presented an optimization model for FL over a wireless network.
Vepakomma *et al.*, [19]	✗	✗	✗	✗	✓	This work presented a split learning framework for healthcare.
Thapa *et al.*, [20]	✗	✗	✗	✗	✓	This work presented a split federated learning framework.
Turina *et al.*, [21]	✗	✗	✗	✗	✓	This work combined federated learning and split learning for enabling privacy preservation.
Liu *et al.*, [22]	✗	✗	✗	✓	✗	This work presented a hierarchical FL framework.
Abad *et al.*, [23]	✓	✗	✗	✓	✗	This work presented a hierarchical FL framework and performed resource optimization.
Wang *et al.*, [24]	✗	✗	✗	✓	✗	This work presented a hierarchical FL framework and performed theoretical convergence analysis.
Qu et al. [29]	✓(i.e., discussed resource optimization)	✗	✗	✗	✗	This work presented a ChainFL framework for FL.
Our paper	✓	✓	✓	✓	✓	N.A

C. Hierarchical FL

Few works [22]–[24] studied hierarchical FL. In [22], the authors presented a client-edge hierarchical FL. Every edge is associated with a set of clients. Clients develop their own local models and share them with the edge servers that do aggregation. The aggregated models are exchanged with the devices for an update after aggregation. For a predetermined number of iterations, this procedure is iterative. Finally, the locally aggregated models at the edge servers are aggregated at a cloud to yield a global FL model. Another work [23] also presented hierarchical FL where edge aggregated models are computed at small cell base stations (SBSs) via iterative interaction between the devices and SBSs. Finally, a global FL model is computed by aggregating edge aggregated models. Wang *et al.* in [24] performed a thorough investigation of the mathematical convergence for hierarchical FL. All the works in [22]–[24] are based on a centralized aggregation server that might cause performance degradation in the case of a centralized server failure. Additionally, there may be fairness issues in the case of hierarchical FL. Some of the groups may have better performance, and thus more influence the global model compared to groups with poor performance. Therefore, we must take into account the fairness issue while designing hierarchical FL schemes. A summary of the comparison of our work with the existing works is given in Table I.

III. Proposed Framework

We provide our proposed HSFL framework in this part, which is depicted in Fig. 1a. The framework consists of a



(a) Proposed framework

(b) Sequence diagram

Fig. 1: HSFL: (a) Proposed framework, (b) sequence diagram

set of resource-constrained IoT devices involved in learning. Generally, the IoT devices perform multiple tasks (e.g., learning, augmented reality signal processing tasks, gaming, etc.) simultaneously. Different devices will have variable computing resources (i.e., CPU-cycles/sec) for performing learning tasks (i.e., local model computation). Some of the devices will be able to compute their models faster compared to the devices with less computing power. Additionally, due to resource constraints, the devices might not be able to compute their complete local learning models. Therefore, the devices will compute a part of their models and offload the remaining learning task to the nearby edge servers. Note that output labels are required for computing a loss function at the edge servers. There can be two possible ways to compute the loss function, such as edge labels-enabled HSFL and device labels-enabled HSFL. For edge labels-enabled HSFL, output labels are available at the edge servers, whereas in device labels-enabled HSFL, the server sends the output of the last layer to the devices for computing a loss function. Here, we assume that edge servers have output labels for computing the loss functions at an edge. The computing capacity of the edge strictly determines the number of devices that can offload their tasks to that edge server. Subsequently, devices per edge server should follow the maximum available computing power at the edge servers. After computing the remaining part of the local models of devices at the edge, the computed task data are shared with the devices for updating their weights. Next, the devices will send the local models to the edge servers for aggregation to produce edge aggregated models, as shown in Fig. 1b:

- *Step 1:* Compute the partial local models at end-devices due to computing resource constraints.
- *Step 2:* Partial local models are offloaded to the edge.
- *Step 3:* Partial local models are learned at the edge.
- *Step 4:* After computing the remaining local models using the partial models and data labels, send the gradients to devices for updating their weights.
- *Step 5:* Send the updated weights of all devices to their associated edge servers, where edge aggregations will take place.
- *Step 6:* Next, the local devices receive the edge aggregated models for further update. For a fixed number of edge iterations, this process iteratively takes place.
- *Step 7:* Next, the edge aggregated models are shared among different edge servers. The transfer of edge aggregated models can be either through core network or wireless. For core network, the delay will be less because of high speed optical fibre link.
- *Step 8:* After edge aggregated models sharing, global model is computed at every edge server in a distributed manner, and thus avoids SPF.

To enable HSFL for a wireless network, one must efficiently allocate wireless and computing resources. Local devices and edge servers can be used to provide computing resources. A local device's energy consumption is proportional to its operating frequency. However, as the local operating frequency rises, the computation time for the local model lowers. As a result, we must carefully balance the device operating frequency and the time required to compute the local model. Meanwhile, there must be an effective learning task offloading scheme for end-devices. The effective criterion for learning task offloading can be transmission latency. Additionally, we must take into account the computing power at both devices and servers while computing local models.

IV. System Model and Problem Formulation

We consider a system that consists of a set $\mathcal{N}$ of N devices deployed randomly. Every device n has a local dataset

TABLE II: Summary of key notations.

Notation	Description
$\mathcal{N}$	Devices set
$\mathcal{M}$	Edge servers-based SBSs set
$\mathcal{R}$	Resource blocks set
R	Total resource blocks
N	Total devices
M	Total SBSs
$x_{n,r}$	Binary resource block allocation variable
$y_{n,m}$	Binary association variable
p_n	Transmit power variable
θ	Relative local accuracy
$c_{n,m}$	Computing power assigned to device n at server m
α	Scaling constant used in cost function
Γ	Throughput
ω_n	Local model weights of device n
ω_s	Edge aggregated model weights
z	Global model weights
$\boldsymbol{X}$	Resource allocation matrix
$\boldsymbol{Y}$	Association matrix
$\boldsymbol{P}$	Power allocation matrix
$\boldsymbol{C}$	Edge server computing resource allocation matrix
χ	Constant that is dependent on local dataset
ϵ	HSFL global accuracy
B	Bandwidth of a resource block
q_n	CPU-cycles required to process a single data point at edge server
v_n	Partial model of device n
f_n	Operating frequency of device n
I_g	Global rounds

$\mathcal{D}_n$ comprising of D_n data points. Meanwhile, there is a set $\mathcal{M}$ of M edge servers-enabled SBSs. Each edge server $m \in \mathcal{M}$ has a certain computing power $\bar{C}_m$ that limits the association of end-devices with it. Table II contains a list of important notations. Depending on their available processing power, the devices compute a portion of their local model before sending the rest to the edge servers. After computation (details are already given in Section III) of local models, an edge aggregated model is obtained. The aggregated models at the edge are then sent back to the devices. For a specific number of edge iterations, this technique is iterative. Next to computing the models aggregated at the edge, sharing of them takes place among different servers. Lastly, the global model is computed. Now, we present the communication model.

A. Communication Model

For communication, we utilize orthogonal frequency division multiple access (OFDMA) in our work. The transmission of DSFL updates between the edge servers and devices uses a collection of $\mathcal{R}$ resource blocks. We suggest using resource blocks that cellular users already utilize. Let the binary variable $x_{n,r}$ denote the allocation of resource blocks to devices (i.e., $x_{n,r} = 1$ when a device n is assigned resource block r and $x_{n,r} = 0$ otherwise). The devices and cellular users will interfere with one another. The devices engaged in HSFL, however, use resource blocks that are orthogonal, preventing interference between them. The size of DSFL local updates is dependent on the architectures and the type of local learning models (e.g., CNN) [31]. Therefore, similar to the works in [30] and [11], we make use of a single block for communication.

$$\sum_{r=1}^{R} x_{n,r} \leq 1, \forall n \in \mathcal{N}. \tag{1}$$

No more than one device may be given access to a particular resource block, i.e.

$$\sum_{n=1}^{N} x_{n,r} \leq 1, \forall r \in \mathcal{R}. \tag{2}$$

For all devices, the allotment of wireless resource blocks must also not be greater than the wireless resources that are readily available, that is,

$$\sum_{r=1}^{R} \sum_{n=1}^{N} x_{n,r} \leq R. \tag{3}$$

When a device uses resource block r, the SINR is provided by:

$$\zeta = \frac{p_n h_{n,r}}{\sum_{t \in \mathcal{T}} p_t h_{t,r} + \sigma_2}, \tag{4}$$

where p_n and $h_{n,r}$ denote the transmit power of the devices and gain of wireless channel, respectively. $\sum_{t \in \mathcal{T}} p_t h_{t,r}$ and σ_2 denote the interference of the cellular users and noise, respectively. A certain range must be followed by the devices transmit power, i.e.,

$$0 \leq p_n \leq P_m, \forall n \in \mathcal{N} \tag{5}$$

In our system, taking the summation of transmit power of devices should follow the following constraint, i.e.,

$$\sum_{n=1}^{N} p_n \leq P_{\max} \tag{6}$$

The device must offload its task to a maximum of one edge server, i.e.,

$$\sum_{n=1}^{M} y_{n,m} \leq 1, \forall n \in \mathcal{N}, \tag{7}$$

where $y_{n,m}$ denotes the variable that shows the association of devices with edge server (i.e., $y_{n,m} = 1$ if device n is associated with edge server m and 0 otherwise). The serving capacity of the edge servers must not be violated.

$$\sum_{n=1}^{N} y_{n,m} \leq C_m, \forall m \in \mathcal{M}, \tag{8}$$

The achievable throughput can be written as:

$$\Gamma(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{P}) = B \log_2(1 + \zeta), \tag{9}$$

where B is the resource block bandwidth that is used the same for all resource blocks in our model. Next, we discuss the wireless HSFL model.

B. HSFL for Wireless Networks

In HSFL, a set $\mathcal{N}$ of N devices is divided into U groups each with N_u devices. Each device n_u of group u has a local dataset of k_u^n data points. By optimizing device weights, HSFL seeks to minimize the global loss function [32]. For instance, the loss function for regression will be different compared to the loss function for image classification tasks. On the other hand, generally, end-devices have limited computing power. Therefore, end-devices will compute a portion of their local models due to computing resources constraints. In addition to learning, the devices have other tasks (e.g., augmented reality) to do. Therefore, the available computing power for performing learning task at device m is equal to $f_n = \bar{f}_n - f_n^{mis}$. Depending on available computing power f_n, a part of the model is learned by the devices and the reaming is learned by the edge server. Let the device n complete a task (i.e., partial computation of local model) by processing e_n computing points with each requires a q_n computing power. The partial local model is sent to the edge server for further computation. Next to transmission of the output (i.e., the partial local model) of device n to edge server m, the computing time (i.e., for processing v_n computing points of the partial local model) at the server can be given by:

$$t_{n,m}(\boldsymbol{C}) = \frac{q_n e_n}{c_{n,m}}, \tag{10}$$

where $c_{n,m}$ and q_n denote the continuous variable that denotes the computing power assigned to learning task of device n by the edge server m and CPU-cycles/sec needed for processing a single data point, respectively. Note that every edge server has a certain computing power to serve the end-users, and therefore, the total computing power allocated by the edge server m should be within a limit. The computing power of the edge servers must take values between the defined range, i.e.,

$$c_{\min} \leq c_{n,m} \leq c_{\max}, \forall m \in \mathcal{M}. \tag{11}$$

Moreover, the overall processing power should be equal to or less than the processing power that is readily available.

$$\sum_{m=1}^{M} \sum_{n=1}^{N} y_{n,m} c_{n,m} \leq C_{\max}. \tag{12}$$

The devices will perform updating the models locally and communicate them to the edge following partial local model computation and gradient sharing with the end-devices. Then the time taken by the device n in partial model computing and weight update after receiving gradients from edge server is given by.

$$t_{n,l}(\boldsymbol{\theta}) = \frac{\log(1/\theta) e_n q_n}{f_n}, \tag{13}$$

where θ denotes the RLA. The lower the value of RLA, the better will be the performance of the local learning model, and vice versa. Note that the local accuracy of devices significantly affects global HSFL rounds while keeping the global accuracy fixed. For the global HSFL accuracy ϵ and relative accuracy θ, we can write [17], [28].

$$I_g(\varepsilon, \boldsymbol{\theta}) = \frac{\chi \log(1/\varepsilon)}{1-\theta}, \tag{14}$$

where χ is a constant that is dependent on the local dataset. It is evident from (14) that for a fixed global HSFL accuracy, a drop in RLA will result in a proportional decrease in global communication rounds, and vice versa. The local accuracy performance is influenced by a number of variables, including the local dataset, local iterations, and the local model design. One can usually state that for a particular local dataset and an architecture, the local iterations will generally determine local model performance. In HSFL, there is a deadline to compute the local model. An increase in local operating frequency is needed in order to increase the local iterations while maintaining the local model computing deadline. The device energy consumption, however, increases when the computation frequency is increased. Meanwhile, every end device has power constraints for backup. As a result, we must balance the RLA performance and the local operating frequency. The square of the local computing resource determines how much energy a device uses [28]. Meanwhile, there are limitations on the available backup power for devices. Therefore, the local operating frequencies of the devices should be selected wisely. Alternatively, to improve local model (i.e., reducing θ) within the allowable time while keeping other parameters fixed, there is a need to increase the local iterations. However, such a raise in local device frequency will increase energy consumption. Therefore, we must make a tradeoff between θ and local energy consumption. The requirement that every device's energy usage must be within the permitted limit can be represented by the notation.

$$\sum_{n=1}^{N} \theta_n \geq O_{\min}, \tag{15}$$

where $O_{\min}$ is dependent mainly on the energy consumption required for training a local model. More local energy is a need for lower values of $O_{\min}$, and vice versa. Also, the θ should be within range, i.e.,

$$\theta_{\min} \leq \theta_n \leq \theta_{\max}. \tag{16}$$

Let u_n denotes the local model output. Then, the transmission latency for sending u_n to the server m at the network edge is given by.

$$t_{n,p}(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{P}) = x_{n,r} y_{n,m} \left(\frac{u_n}{\Gamma_n} \right). \tag{17}$$

The transmission energy can be given by:

$$e_{n,p}(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{P}) = x_{n,r} y_{n,m} \left(\frac{u_n p_n}{\Gamma_n} \right). \tag{18}$$

A summary of the HSFL algorithm is given in Algorithm 1. Initially, all devices compute their partial local models due to computing resource constraints and send them to the edge server for further computation (lines 6-8). Send the computed model parameters back to devices for updating their

Algorithm 1 HSFL Algorithm

1: Initialization of weights ω_0
2: **for** i=1,2,..., Global Rounds **do**
3: ***Step 1: For all devices, run parallel***
4: **for** k = 1, 2,..., edge iterations **do**
5: **for** i= 1, 2,..., Local iterations **do**
6: Compute partial local models for devices.
7: Send partial local models to edge servers.
8: Compute partial models at edge servers.
9: Share the gradients of edge servers with the devices to update their local weights.
10: $\omega_u^n(i) \longleftarrow \omega_u^n(i-1) - \eta \nabla F_n(\omega_u^n(i-1))$
11: Send the local weights to the edge servers.
12: **end for**
13: $\omega_u^s(k) \longleftarrow Edge\ aggregation(\omega_u^n)$
14: $\omega_u \longleftarrow \omega_u^s$
15: **end for**
16: ***Step 2: Global aggregation***
17: $\omega_g(k) \longleftarrow Global\ aggregation(\omega_u)$
18: $\omega_u^n \longleftarrow \omega_g$
19: **end for**

weights (line 9). Then, the local learning model weights are updated (line 9 and line 10). After computing local learning models for all devices, edge aggregated models are computed (line 13). The computed edge aggregated models are shared among all edge servers. In the end, the computation of a global model takes place at every edge (line 17).

C. Problem Formulation

First, we define the cost that consider both latency and energy. The latency in our model is due to local model computing latency, transmission latency, and edge server computing latency. The delay due to computing partial local model at edge server is given by:

$$T_{\text{edge}}(\boldsymbol{C}) = \sum_{m=1}^{M}\sum_{n=1}^{N} y_{n,m} t_{n,m}. \tag{19}$$

For an edge aggregated model, the latency is given by [17].

$$T_{\text{trans}}(\boldsymbol{\theta}, \boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{P}, \boldsymbol{C}) = \sum_{m=1}^{M}\sum_{n=1}^{N}\left(\frac{t_{n,p}}{1-\theta}\right). \tag{20}$$

By applying Taylors' approximation, (20) can be rewritten as.

$$T_{\text{trans}}(\boldsymbol{\theta}, \boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{P}, \boldsymbol{C}) = (1+\theta)\sum_{m=1}^{M}\sum_{n=1}^{N}\left(t_{n,p}\right). \tag{21}$$

The total latency cost for HSFL is the local model computation latency at the edge and T_{trans}.

$$T_{\text{HSFL}}(\boldsymbol{\theta}, \boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{P}, \boldsymbol{C}) = T_{\text{trans}} + T_{\text{edge}}. \tag{22}$$

The energy cost accounts for the transmission energy. Similar to (21), the cost associated with transmission energy can be rewritten as.

$$E_{\text{HSFL}}(\boldsymbol{\theta}, \boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{P}, \boldsymbol{C}) = (1+\theta)\frac{x_{n,r} u_n p_n}{\Gamma_n}. \tag{23}$$

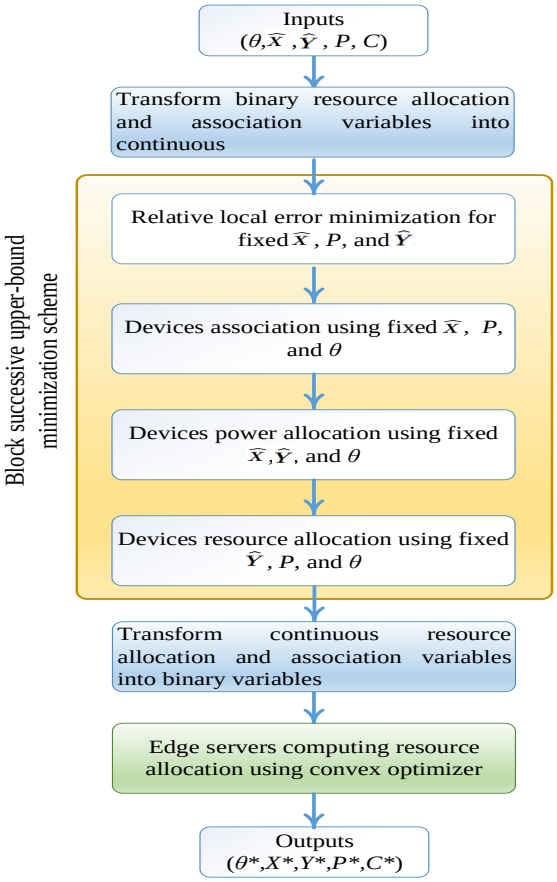


Fig. 2: Proposed solution approach.

As a whole, we write the cost of HSFL as

$$C_{\text{HSFL}}(\boldsymbol{\theta}, \boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{P}, \boldsymbol{C}) = \alpha E_{HSFL} + (1-\alpha) T_{HSFL}. \tag{24}$$

where $\alpha \in (0, 1)$ is the scaling constant that scales the proportion of latency and energy. Note that θ depends mainly on the local model architecture and local dataset distribution. Next, we present our formulated problem.

$$\mathbf{P-1}: \underset{\theta, \mathbf{X}, \mathbf{Y}, \mathbf{P}, \mathbf{C}}{\text{minimize}} \; C_{\text{HSFL}}(\boldsymbol{\theta}, \boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{P}, \boldsymbol{C}) \tag{25}$$

subject to:

$$(1)-(3), (5)-(8), (11)-(12), (15), (16). \tag{25a}$$

The problem **P-1** is a MINLP problem. Moreover, problem **P-1** is NP-Hard. A problem is NP if its worst-case running time is more than $O(n^k)$, where n and k are input size and some constant, respectively. The problem is NP-Hard if we can translate the algorithm to solve that given problem into those used to solve any NP-problem. NP-hard means "*at least as hard as any NP-problem*". Specifically, in NP-Hard problems, it is hard to find a solution but can be solved using non-deterministic machines. Finding the solution to our problem **P-1** is very hard but one can solve it after making it simple and applying various schemes. It has three continuous variables, such as relative local frequency matrix, devices transmit power

matrix $\boldsymbol{P}$, and edge server computing capacity matrix $\boldsymbol{C}$. The other two variables, such as resource allocation and association are binary variables.

V. Problem Decomposition and Solution

A. Problem Decomposition

Problem **P-1** has five variables, i.e., transmit power variable $\boldsymbol{P}$, relative accuracy variable $\boldsymbol{\theta}$, association variable $\boldsymbol{Y}$, edge computing resource variable $\boldsymbol{C}$, and resource allocation variable $\boldsymbol{X}$. $\boldsymbol{\theta}$ has a nature that varies with many parameters, local model architecture, local dataset quality (e.g., noise-less nature), and local dataset distribution. Problem **P-1** is difficult to solve because of the MINLP nature and it becomes NP-Hard for a large number of devices, resource blocks, and edge servers. Therefore, we cannot apply convex optimization schemes. To solve **P-1**, we first relax the binary variables into continuous variables. Then, **P-1** can be rewritten as.

$$\mathbf{P-2}: \underset{\theta,\widehat{X},\widehat{Y},\mathbf{P},\mathbf{C}}{\text{minimize}} \; C_{\text{HSFL}}(\theta,\widehat{\boldsymbol{X}},\widehat{\boldsymbol{Y}},\boldsymbol{P},\boldsymbol{C}) \tag{26}$$

subject to:

$$\text{C1}: \sum_{r=1}^{R} \widehat{x}_{n,r} \leq 1, \forall n \in \mathcal{N},$$

$$\text{C2}: \sum_{n=1}^{N} \widehat{x}_{n,r} \leq 1, \forall r \in \mathcal{R},$$

$$\text{C3}: \sum_{r=1}^{R}\sum_{n=1}^{N} x_{n,r} \leq R,$$

$$\text{C4}: 0 \leq p_n \leq P_m, \forall n \in \mathcal{N},$$

$$\text{C5}: \sum_{n=1}^{N} p_n \leq P_{\max},$$

$$\text{C6}: \sum_{n=1}^{M} \widehat{y}_{n,m} \leq 1, \forall n \in \mathcal{N},$$

$$\text{C7}: \sum_{n=1}^{N} y_{n,m} \leq C_m, \forall m \in \mathcal{M},$$

$$\text{C8}: \sum_{m=1}^{M}\sum_{n=1}^{N} y_{n,m}c_{n,m} \leq C_{\max},$$

$$\text{C9}: \sum_{n=1}^{N} \theta_n \geq O_{\min},$$

$$\text{C10}: \theta_{\min} \leq \theta_n \leq \theta_{\max},$$

$$\text{C11}: c_{\min} \leq c_{n,m} \leq c_{\max}, \forall m \in \mathcal{M},$$

$$\text{C12}: 0 \leq \widehat{x}_{n,r} \leq 1, \forall n \in \mathcal{N}, r \in \mathcal{R},$$

$$\text{C13}: 0 \leq \widehat{y}_{n,m} \leq 1, \forall n \in \mathcal{N}, m \in \mathcal{M}, .$$

In **P-2**, the variables $\widehat{x}_{n,m}$ and $\widehat{y}_{n,m}$ are continuous version of the binary variables $x_{n,m}$ and $y_{n,m}$, respectively. Because of the non-convex character, the problem remains highly complex and challenging to resolve even after converting them to continuous variables. As a result, we divide the core problem into sub-problems, as depicted in Fig. 2. We first decompose the problem **P-2** into two sub-problems: (a) jointly optimizing transmit power, resource allocation, association, as well as RLA, and (b) optimization of edge server computing resource allocation. The problem for optimizing association, power allocation, resource allocation, and RLA can be written as.

$$\mathbf{P-3}: \underset{\theta,\widehat{X},\widehat{Y},\mathbf{P}}{\text{minimize}} \; C_{\text{HSFL}}(\boldsymbol{\theta},\widehat{\boldsymbol{X}},\widehat{\boldsymbol{Y}},\boldsymbol{P}) \tag{27}$$

subject to

$$(C1)-(C10),(C12),(C13). \tag{27a}$$

Problem **P-3** is a non-convex programming problem. Additionally, the problem has still four continuous variables that are difficult to solve directly. Also, it can not be solved using conventional convex optimization schemes. Therefore, there is a need to transform it into a convex problem and then apply convex optimization for a solution. Another possible way can be to use a scheme that is well-suitable for non-convex problems (e.g., BSUM). Next, we define the edge computing resource allocation sub-problem as follows.

$$\mathbf{P-4}: \underset{\mathbf{C}}{\text{minimize}} \; C_{\text{HSFL}}(\boldsymbol{C}) \tag{28}$$

subject to:

$$(C8),(C11). \tag{28a}$$

B. Proposed Solution

Here, solutions of various sub-problems are discussed. First, we solve the sub-problem **P-4** which is edge server computing resource allocation problem. To solve the convex sub-problem **P-4**, we can use a convex optimizer.

Lemma 1. (Convexity of Sub-problem **P-4**)*: We use a Hessian matrix for proving the convexity of **P-4** for all possible values of the variable c. For **P-4**, the values of the Hessian matrix can be computed as follows [33], [34].*

$$\gamma_{i,j} = \frac{\partial^2 C_{HSFL}(\boldsymbol{C})}{\partial c_i \partial c_j}, \forall i = 1,2,...,M, \;\; j = 1,2,...,M \tag{29}$$

$C_{HSFL}(\boldsymbol{C})$ consists of a summation of terms with c_i $i = 1,2,...,M$. Taking partial derivative of the terms other than diagonal terms will result in 0. From (29), we will get a $M \times M$ diagonal matrix $\boldsymbol{\gamma}$ with each diagonal element $\frac{2v_n q_n}{c_m^3}$, where c_m denotes the computing resources assigned by edge server m to its associated devices. The positive semidefinite can be given as follows.

$$C^T \boldsymbol{\gamma} C \geq 0, \tag{30}$$

*For all possible positive values (i.e., between $c_m$in and $c_m$ax) of the variable c, it is clear that (30) is fulfilled. This shows that the objective function of **P-4** is a convex function. Additionally, the constraints are linear inequality constraints. Therefore, sub-problem **P-4** is a convex optimization problem.*

Now, we present the solution of sub-problem **P-3**, which is a non-convex problem. Therefore, one can not solve it directly using a convex optimization solver. There are two

Algorithm 2 BSUM Algorithm

1: **Initialization Phase:** Set $k = 0$, $\epsilon_1 > 0$. Then, initial feasible solutions are computed, $(\boldsymbol{\theta^{(0)}}, \mathbf{X}^{(0)}, \mathbf{Y}^{(0)}, \mathbf{P}^{(0)})$;
2: **repeat**
3: Use index set $\mathcal{I}^k$;
4: Let $\boldsymbol{\theta}_i^{(k+1)} \in \min_{\boldsymbol{\theta}_i \in \theta} C_i\big(\boldsymbol{\theta}_i; \boldsymbol{\theta}^{(k)}, \mathbf{X}^{(k)}, \mathbf{Y}^{(k)}, \mathbf{P}^{(k)}\big)$;
5: Compute $\theta_j^{(k+1)} = \theta_j^k, \forall j \notin \mathbf{I}^k$;
6: Find $\mathbf{X}_i^{(k+1)}$, $\mathbf{Y}_i^{(k+1)}$, and $\mathbf{P}_i^{(k+1)}$, by solving (38), (40), and (41);
7: $k = k + 1$;
8: **until** $\| \frac{C_i^{(k)} - C_i^{(k+1)}}{C_i^{(k)}} \| \leq \epsilon_1$
9: Then, set $\big(\mathbf{X}_i^{(k+1)}, \mathbf{Y}_i^{(k+1)}, \mathbf{P}_i^{(k+1)}\big)$ as the desired solution.

different ways to solve **P-3**, such as (a) transforming a non-convex problem into a convex and (b) using an approach that works well for a non-convex problem **P-3**. Transforming the non-convex problem into a convex problem is very difficult, and therefore, we solve **P-5** using a scheme that is suitable for non-convex problems. Here, our scheme is based on the modification of the BSUM presented [25]. BSUM solves the problem by using distributed computation in parallel. BSUM enables us with the features of easier problem decomposability and fast convergence[35]. BSUM can be given as follows.

$$\min_{\boldsymbol{v}} g(\boldsymbol{v}_1, \boldsymbol{v}_2, ..., \boldsymbol{v}_o), s.t. \boldsymbol{v}_0 \in \mathcal{A}_o, \forall o \in O, o = 1, 2, .., O, \tag{31}$$

where $\mathcal{A} := \mathcal{A}_1 \times \mathcal{A}_2 \times ..., \mathcal{A}_O$. Moreover, the nature of $g(\cdot)$ is continuous. $\boldsymbol{v}_o$ and $\mathcal{A}_0$ are a block of variables and a closed convex set, respectively. To solve a single block of variables, one can use a BCD scheme as follows.

$$\boldsymbol{v}_o^t \in \underset{\boldsymbol{v}_o \in \mathcal{A}_j}{\operatorname{argmin}} g(\boldsymbol{v}_o, \boldsymbol{v}_{-o}^{t-1}), \tag{32}$$

where $\boldsymbol{v}_{-o}^{t-1} := (v_1^{t-1}, ..., v_{o-1}^{t-1}, v_{o+1}^{t-1}, ..., v_o^{t-1}), \boldsymbol{v}_k^t = \boldsymbol{v}_k^{t-1}$ for $o \neq k$. It is very difficult to get a solution of (37) and (38) by applying BCD scheme. The reason for this is the non-convex nature. Addressing this challenge, our work considers BSUM, which uses minimizing the upper-bound function $b(\boldsymbol{v}_o, \boldsymbol{y})$ of $g(\boldsymbol{v}_o, \boldsymbol{y}_{-o})$. One can consider different upper-bound functions (e.g., Jensen's upper-bound and quadratic upper-bound) for BSUM. We made the following assumption similar to the previous works using BSUM [25], [35].

Assumptions:
1) $b(\boldsymbol{v}_o, \boldsymbol{y}) = g(\boldsymbol{y})$,
2) $b(\boldsymbol{v}_o, \boldsymbol{y}) > g(\boldsymbol{v}_o, \boldsymbol{y}_{-o})$,
3) $b'(\boldsymbol{v}_o, \boldsymbol{y}; \boldsymbol{q}_o)|_{\boldsymbol{v}_o = \boldsymbol{y}_o} = g'(\boldsymbol{y}; \boldsymbol{q}), \boldsymbol{y}_o + \boldsymbol{q}_o \in \mathcal{A}_o$,

To guarantee the global upper-bound nature of b, assumptions (1) and (2) are used [25], [35]. These assumptions are made since BSUM is based on minimizing the upper-bound to give a good approximate solution to non-convex problems [25]. This is the reason assumptions 1 and 2 are made to keep the b as upper-bound. Also, the upper-bound function steps are negative of the objective function gradient in the direction of q, as indicated by assumption 3. For an upper-bound function, here, quadratic penalization is adopted, i.e.,

$$b(\boldsymbol{v}_o, \boldsymbol{y}) = g(\boldsymbol{v}_o, \boldsymbol{y}_{-o}) + \frac{\mu}{2} \| \boldsymbol{v}_o - \boldsymbol{y}_o \|^2, \tag{33}$$

where μ denotes the positive penalty constant. Then, the proximal upper-bound function can be solved as follows.

$$\begin{cases} \boldsymbol{v}_o^t \in \min\limits_{\boldsymbol{v}_o \in \mathcal{A}_j} h(\boldsymbol{v}_o, \boldsymbol{v}_o^{t-1}), \forall o \in O, \\ \boldsymbol{v}_k^t = \boldsymbol{v}_k^{t-1}, \forall k \notin O. \end{cases} \tag{34}$$

In BSUM, successive updating blocks of primary variables are performed to minimize the upper-bound function of the original objective function. A block of variables should reach a minimum point $\boldsymbol{v}^* = \boldsymbol{v}_j^{t+1}$ for a stationary solution to be coordinate-wise minimum.

Remark 1 (Algorithm 2 convergence): Using sub-linear fashion, $O(\log(1/\epsilon))$ is the highest number of iterations, algorithm2 will take to converge to ϵ-optimal solution [36].

P3 using BSUM can be written as

$$\min_{\theta \in \theta \widehat{\mathbf{X}} \in \widehat{\mathcal{X}}, \widehat{\mathbf{Y}} \in \widehat{\mathcal{Y}}, \mathbf{P} \in \mathcal{P}} C(\theta, \widehat{\mathbf{X}}, \widehat{\mathbf{Y}}, \mathbf{P}), \tag{35}$$

where $C(\widehat{\theta}, \widehat{\mathbf{X}}, \widehat{\mathbf{Y}}, \widehat{\mathbf{P}}) = C_{\text{HSFL}}(\widehat{\boldsymbol{\theta}}, \widehat{\boldsymbol{X}}, \widehat{\boldsymbol{Y}}, \widehat{\boldsymbol{P}})$. Furthermore, the feasible of sets of $\widehat{\theta}$, $\widehat{\mathbf{X}}$, $\widehat{\mathbf{Y}}$, and $\widehat{\mathbf{P}}$ are:

$$\theta \triangleq \{\theta_n : \theta_{\min} \leq \theta_{\max}, \forall n \in \mathcal{N}, \sum_{n=1}^{N} \theta_n \geq O_{\min}\}.$$

$$\widehat{\mathcal{X}} \triangleq \{\widehat{\mathbf{X}} : \sum_{n \in \mathcal{N}} \widehat{x}_{n,r} \leq 1, \forall r \in \mathcal{R}, \sum_{r \in \mathcal{R}} \widehat{x}_{n,r} \leq 1, \forall r \in \mathcal{R}, \widehat{x}_{n,r} \in \{0, 1\}, \forall n \in \mathcal{N}, r \in \mathcal{R}\},$$

$$\widehat{\mathcal{Y}} \triangleq \{\widehat{\mathbf{Y}} : \sum_{n=1}^{N} \widehat{y}_{n,m} \leq 1, \forall m \in \mathcal{M}, \sum_{n=1}^{N} \sum_{m=1}^{M} \widehat{y}_{n,m} c_{n,m} \leq C_{\max}, \forall n \in \mathcal{N}, \forall m \in \mathcal{M}, \widehat{y}_{n,m} \in \{0, 1\} \forall n \in \mathcal{N}, \forall m \in \mathcal{M}\},$$

$$C_i(\widehat{\theta}_i; \theta^k, \widehat{\mathbf{X}}^k, \widehat{\mathbf{Y}}^k, \mathbf{P}^k) = C(\theta_i; \tilde{\theta}, \tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{P}}) + \frac{\mu_i}{2} \| (\theta_i - \tilde{\theta}) \|^2 . \tag{36}$$

Similar to above, one can use penalty term to $\widehat{\mathbf{X}}_i$, $\widehat{\mathbf{Y}}_i$, and $\mathbf{P}_i$. b with respect to θ_i, $\widehat{\mathbf{X}}_i$, $\widehat{\mathbf{Y}}_i$, and $\mathbf{P}_i$ in (36) produces unique , $\tilde{\mathbf{X}}$, $\tilde{\mathbf{Y}}$, and $\tilde{\mathbf{P}}$ in every iteration. These unique values will be used as a solution of $(k-1)$ iteration. For $(k+1)$ iteration, the solution can be given by:

$$\theta_i^{(k+1)} \in \min_{\theta_i \in \theta} C_i\Big(\theta_i; \theta^{(k+1)}, \widehat{\mathbf{X}}^{(k+1)}, \widehat{\mathbf{Y}}^{(k+1)}, \mathbf{P}^{(k+1)}\Big), \tag{37}$$

$$\widehat{\mathbf{X}}_i^{(k+1)} \in \min_{\widehat{\mathbf{X}}_i \in \widehat{\mathcal{X}}} C_i\left(\widehat{\mathbf{X}}_i; \theta^{(k+1)}, \widehat{\mathbf{X}}^k, \widehat{\mathbf{Y}}^{(k+1)}, \mathbf{P}^{(k+1)}\right), \tag{38}$$

$$\widehat{\mathbf{Y}}_i^{(k+1)} \in \min_{\widehat{\mathbf{Y}}_i \in \widehat{\mathcal{Y}}} C_i\left(\widehat{\mathbf{Y}}_i; \theta^{(k+1)}, \widehat{\mathbf{X}}^{(k+1)}, \widehat{\mathbf{Y}}^k, \mathbf{P}^{(k+1)}\right), \tag{39}$$

$$\mathbf{P}_i^{(k+1)} \in \min_{\mathbf{P}_i \in \mathcal{P}} C_i\left(\mathbf{P}_i; \theta^{(k+1)}, \widehat{\mathbf{X}}^{(k+1)}, \widehat{\mathbf{Y}}^{(k+1,)}, \mathbf{P}^k\right), \tag{40}$$

The above are the blocks for a certain parameter ($\boldsymbol{\theta}$) while keeping the others ($\widehat{\boldsymbol{X}}, \widehat{\boldsymbol{Y}}, \boldsymbol{P}$) fixed. For solving (37)-(40), we use our proposed BSUM *Algorithm* 2. After getting a convergence using algorithm 2, the resource allocation and offloading/association variables are transformed back to binary.

C. Complexity and Convergence Analysis

1) Complexity Analysis: In our solution approach, we have three sub-problems. For edge server resource and local computing resource allocation sub-problems, we use a convex optimizer that converges fast within a finite number of iterations [31]. Therefore, one can say that local computing resource and edge server computing resource allocation sub-problems have a reasonable complexity. On the other hand, for the joint wireless resource, transmit power, and association sub-problem, we employ a BSUM-based approach that also has a reasonable complexity. The standard BSUM in Algorithm 2 is based on minimizing the proximal upper bound. To minimize the upper-bound function, every block of variables is updated in an iterative manner. Until the upper-bound function converges to a coordinate-wise minimal stationary point, variable blocks are updated iteratively. Using Remark 1 and assumptions, it is clear that BSUM in Algorithm 2 has a sub-linear convergence rate. Additionally, BSUM in Algorithm 2 uses flexible update rules. From the above discussion, it is clear that Algorithm 2 has a sub-linear iteration complexity $O(1/i)$ for the i^{th} iteration [36]. Note that sub-linear means that algorithm converges different (i.e., faster or slower) compared to linear time complexity.

2) Convergence Analysis: We discuss the convergence analysis of the proposed HSFL scheme. There are two main aspects of convergence in proposed HSFL: (a) BSUM and convex optimization-based solution convergence and (b) HSFL convergence in terms of upper bound. From remark 1, it is clear that proposed BSUM algorithm convergence with a reasonable complexity. Additionally, convex optimization-based solutions for local computing resource management and edge server computing resource management also converges within finite iterations. Therefore, one can say that BSUM and convex optimization based solution converges within finite, reasonable iterations. For analyzing convergence of HSFL, we made several assumptions [24], [27], [28], [30], [31] as

- $F(\omega_g)$ is a convex function.
- $F(\omega_g)$ has $\rho-$Lipschitz nature.

$$\| \nabla(F(\omega_{g(t+1)})) - \nabla(F(\omega_{g(t)})) \| \leq \rho \| \omega_{g(t+1)} - \omega_{g(t)} \| \tag{41}$$

  where ρ is positive constant, whereas $\| * \|$ represent norm of $*$.
- $F(\omega_g)$ has a β-smooth nature.

$$\| \nabla F_i(\omega_{g(t+1)}) - \nabla F_i(\omega_{g(t)}) \| \leq \beta \| \omega_g - \omega_g' \| \quad \text{for any } \omega_{g(t+1)},\ \omega_{g(t)} \tag{42}$$

Similar to the works in [27] and [22], one can use the notion of auxiliary parameter vector $v_{[k]}(t)$ that is based on centralized gradient descent. Here, we observe the convergence for both traditional FL and HSFL. We can divide the whole local T iterations into K intervals and each interval can be represented by $t \in [(k-1)\tau, k\tau]$. For FL, within each interval, the local model weights $\omega(t)$ of each device has no divergence from the $v_{[k]}(t)$ for first two iterations [22], [27]. However, with an increase in the number of local iterations (e.g., 4 local iterations), there will be a divergence between $\omega_u^n(t)$ and $v_{[k]}(t)$. The convergence bound of FL depends on the gap between $\omega_u^n(t)$ and $v_{[k]}(t)$. For FL, the convergence upper bound can be given by [27]:

$$F(w^f) - F(w^*) \leq \frac{1}{T\left(\eta\phi - \frac{\rho h(\tau)}{\tau \epsilon_o^2}\right)}, \tag{43}$$

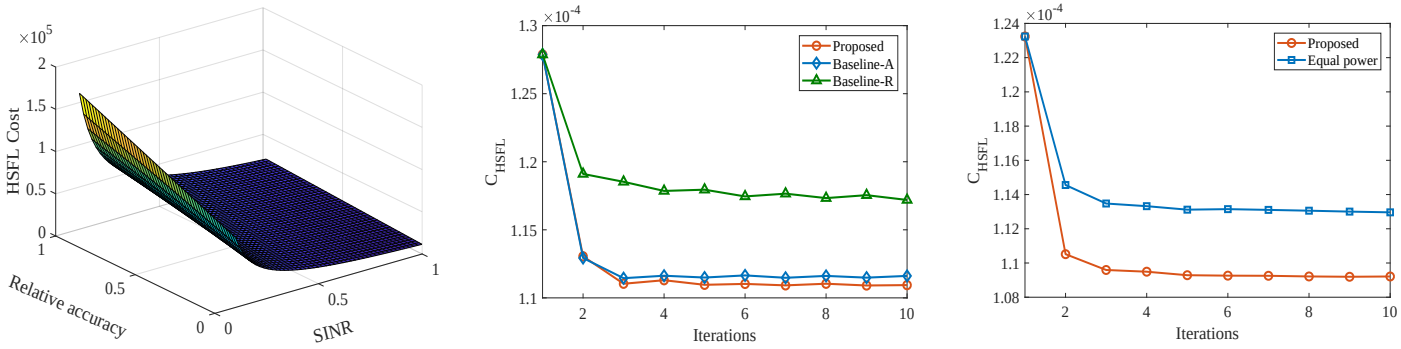
where $h(x) = \frac{\sigma}{\beta}\left((\eta\beta+1)^x - 1\right) - \eta\sigma_x$. w^f and w^* denote the weights after end of learning process and optimal weights, respectively. σ and η denote the client-edge divergence and step size, respectively. On the other hand, for HSFL there can be two kinds of gradients divergences: (a) between devices and edge servers and (b) between edge server and cloud running global aggregators. Therefore, we must carefully tune the edge aggregations, local iterations, and global aggregations for HSFL. For HSFL, similar to hierarchical FL, the upper bound can be given by [22].

$$F(w^f) - F(w^*) \leq \frac{1}{T\left(\eta\phi - \frac{\rho G_c(k_1 k_2, \eta)}{k_1 k_2 \epsilon_o^2}\right)}, \tag{44}$$

where $G_c(k_1 k_2, \eta) = h(k_1 k_2, \Delta, n_q) + \frac{1}{2}(k_2^2 + k_2 + 1)(k_1 + 1)h(k_1.\delta, \eta_q)$. k_1 and k_2 denote number of local iterations prior to edge aggregation and edge aggregations prior to global aggregation. Δ represents the edge-cloud divergence.

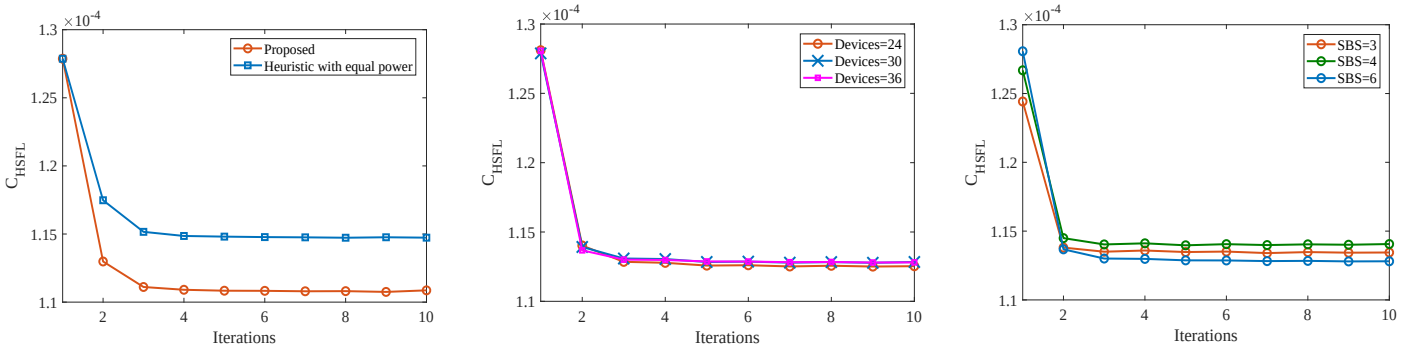
Remark 1. Convergence for IID data*: For IID data, there will be no divergence of devices weights iterations from the virtually centralized weights iterations, and thus $G_c(k)$ will be zero because δ and Δ are both equal to 0 for IID distribution. This shows that the centralized weights iterations are same as distributed weights iterations.*

Remark 2. Convergence for Non-IID data: *For non-IID data, there will be weights divergences such as, client-edge and edge-cloud. In $G_c(k_1 k_2)$, the first term $h(k_1 k_2, \Delta, n_q)$ is due to edge-cloud divergence, whereas the second term $\frac{1}{2}(k_2^2 + k_2 + 1)(k_1 + 1)h(k_1.\delta, \eta_q)$ is due to the client-edge*



(a) (b) (c)

Fig. 3: (a) C_{HSFL} for various values of SINR and RLA , (b)C_{HSFL} vs. iterations for various schemes using 48 devices and 6 SBSs, (c) C_{HSFL} vs. iterations for proposed and heuristic algorithm with equal power using 48 devices and 6 SBSs.



(a) (b) (c)

Fig. 4: (a) C_{HSFL} vs. iterations for proposed and proposed with equal power, (b) C_{HSFL} vs. iterations for proposed scheme using different number of devices, and (c)C_{HSFL} vs. iterations for proposed scheme using different number of SBSs

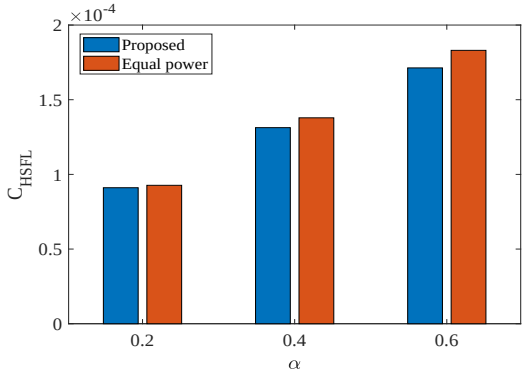


Fig. 5: C_{HSFL} for various values of α.

TABLE III: Simulation Parameters [37], [38].

Simulation Parameter	Value
Devices	50
Cellular users	50
Area	$1000 \times 1000m^2$
Devices transmit power	23 dBm
Frame Structure	Type 1 (FDD)
Bandwidth of resource block (W)	180 kHz
Thermal noise for 1 Hz at 20°C	-174 dBm
Carrier frequency (f)	2 GHz
Sub carriers per resource block	12

divergence. The first and the second terms have exponential relations with k_1, k_2 and k_1, respectively. (44) clearly shows that for a fixed product of $k_1 \times k_2$, an increase in number of edge aggregations (i.e., k_2) will improve performance. However, this will be at the cost of communication resources. Therefore, one must make a tradeoff between the frequency of edge aggregations and performance.

VI. Performance Evaluations

We discuss the simulation outcomes for the suggested HSFL scheme's performance improvement in this section. An area of $1,000 \times 1,000$ is considered with randomly deployed cellular users and set of devices used in HSFL. Table III provides additional simulation-related parameters. The devices involved in HSFL reuse the resource blocks of cellular users and thus, receives interference from them. For HSFL, we deploy multiple SBS that are positioned randomly in the considered for each run. Cellular users transmit signals with equal power in all runs. The figures were all calculated using an average of 35 runs each. Additionally, we compare performance using two baselines, such as baseline-A and baseline-R. Baseline-A differs from the proposed scheme only by random resource allocation, whereas baseline-R differs from the proposed scheme by random association.

Variations in HSFL cost are shown in Fig. 3a for different SINR and RLA values. A low value of RLA and high value

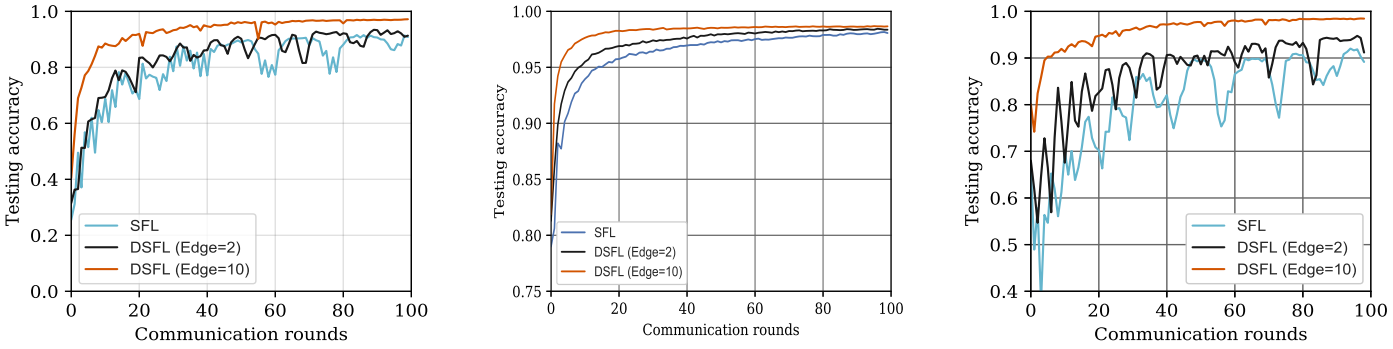


Fig. 6: (a) Accuracy vs. global rounds for non-IID1 using 5 groups with each having 10 devices, (b) accuracy vs. global rounds for non-IID2 using 5 groups with each having 10 devices(c) accuracy vs. global rounds for non-IID3 using 5 groups with each having 10 devices.

of SINR is desirable. However, this will be at the cost of computing and communication cost. The communication cost will be related to resource blocks and transmit power allocation, whereas the computing cost will be related to local model iterations for a fixed local model and architecture. The cost C_{HSFL} vs. iterations for different schemes, including suggested, baseline-A, and baseline-R, are depicted in Fig. 3b. The proposed system performs better than baseline-A and baseline-R, as shown in Fig. 3b. The proposed scheme optimizes RLA, edge server computing resource optimization, and joint resource allocation, association, and power allocation, which is the cause of this performance improvement. Baseline-A performs better than baseline-R, but less well than the proposed approach. The reason for the performance enhancement of baseline-A compared to baseline-R is a dependency of cost C_{HSFL} more on the association compared to resource allocation in the current settings. In the proposed scheme, we perform optimization of various variables: resource allocation, edge computing resource, relative local accuracy, association, and transmit power allocation. We use baselines: Baseline-A (i.e., uses proposed scheme using proposed association, resource allocation, computing resource allocation, and transmit power allocation with random resource allocation) and baseline-R (i.e., uses proposed scheme using proposed association, resource allocation, computing resource allocation, and transmit power allocation with random association) for comparison. Therefore, to see the effect of the proposed transmit power optimization scheme, resource allocation, and association, we compare the performance of the proposed scheme with an exhaustive search algorithm for association and allocation along with equal power. Fig. 3c compares the performance of the proposed scheme with an exhaustive search algorithm using equal power [25]. Fig. 3c reveals that HSFL outperforms an exhaustive search scheme with equal power. Onwards, the cost C_{HSFL} denotes the cost that accounts for transmission latency and transmission energy. Next, Fig. 4a shows the cost C_{HSFL} vs. iterations for the proposal and suggested plan with equal force. Fig. 4a depicts that the proposed scheme outperforms the proposed scheme with equal power. Also, if we increase the maximum limit of maximum power, the performance of the proposed scheme may be further enhanced. Therefore, it is necessary that we should make a tradeoff between the maximum limit of transmit power for all devices. Fig. 4b shows the cost vs. iterations for various devices. It is clear that the proposed scheme converges fast for all numbers of devices (i.e., 24, 30, and 36). Another Fig. 4c shows cost vs. iterations for fixed 24 devices and different SBSs. Similar to Fig. 4b, Fig. 4c converges for different numbers of SBSs. Fig. 5 shows cost C_{HSFL} for variations in the constant α for the proposed scheme and proposed scheme with equal power. Fig. 5 shows a slight increase in cost for higher values of α for both the proposed scheme and the proposed one with equal power. This shows that cost C_{HSFL} has more contributions from the latency term compared to the transmission energy term.

Now, we present learning results for evaluating the performance of the proposed HSFL scheme. We consider MNIST and Fashion MNIST datasets for evaluation. The training data is divided into shards, each of 300 images. We consider three cases, such as non-IID1, non-IID2, and nonIID3. For non-IID1, a single shard is assigned to every device, whereas for non-IID2 two shards are assigned to each device. 5 groups, each with 5 devices are considered. For non-IID3, every device in a group of 10 is assigned images of a single class such that all devices in a group should be assigned all classes of MNIST. For both Figs. 6a, 6b, and 6c, a product of local iterations and edge aggregations per global round of communication is taken 10. For traditional, local iterations equal to 10 is used for Figs. 6a, 6b, and 6c. For Fig. 6a uses single shard per device, whereas for Fig. 6b two shards per device are used. It is clear from Figs. 6a, 6b, and 6c that HSFL outperforms SFL. An increase in number of edge aggregations can improve performance but at the cost of communication. Therefore, one must make a tradeoff between communication cost and performance. Note that computing an edge aggregated HSFL model can offer the advantage of easy reuse of communication resource blocks. Within a small group, one might reuse communication resources more efficiently compared to the case of a single group used for computing SFL model. For the single-use case of SFL, the area will be more. Therefore, to communicate with a single centralized aggregation, devices must transmit with high transmission power that can cause a higher interference to cellular users.

VII. Conclusions

In this paper, we have presented a novel framework, namely, HSFL, that combines split learning and hierarchical federated learning. A problem considering latency and energy is formulated. We subdivided the main problem into two subproblems because the formulated problem was non-convex and NP-hard. Edge server computing resource allocation subproblems is solved using a convex optimization scheme. We employed a BSUM-based strategy for the combined RLA minimization, resource allocation, association, and transmit power allocation. Additionally, discussed the complexity and convergence analysis of the proposed solution. The results were then provided for the various scheme's validation. We came to the conclusion that a variety of IoT applications where devices lack the power to compute their local models could benefit from the use of our HSFL model. Our work can be used as guidelines for various applications where the end-devices will have resource constraints, such as intelligent transportation, healthcare, and Industry 4.0, among others. Our framework offers privacy-aware learning with fast convergence for resource constraints end-devices and is thus preferable for use in future applications.

References

[1] L. U. Khan, Z. Han, W. Saad, E. Hossain, M. Guizani, and C. S. Hong, "Digital twin of wireless systems: Overview, taxonomy, challenges, and opportunities," *IEEE Communications Surveys & Tutorials*, 2022.

[2] K. David and H. Berndt, "6g vision and requirements: Is there any need for beyond 5g?" *IEEE vehicular technology magazine*, vol. 13, no. 3, pp. 72–80, 2018.

[3] L. U. Khan, Z. Han, D. Niyato, E. Hossain, and C. S. Hong, "Metaverse for wireless systems: Vision, enablers, architecture, and future directions," *arXiv preprint arXiv:2207.00413*, 2022.

[4] L. U. Khan, M. Guizani, D. Niyato, A. Al-Fuqaha, and M. Debbah, "Metaverse for wireless systems: Architecture, advances, standardization, and open challenges," *arXiv preprint arXiv:2301.11441*, 2023.

[5] S. He, K. Shi, C. Liu, B. Guo, J. Chen, and Z. Shi, "Collaborative sensing in internet of things: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, 2022.

[6] L. Kong, J. Tan, J. Huang, G. Chen, S. Wang, X. Jin, P. Zeng, M. Khan, and S. K. Das, "Edge-computing-driven internet of things: A survey," *ACM Computing Surveys*, vol. 55, no. 8, pp. 1–41, 2022.

[7] S. Ali, W. Saad, N. Rajatheva, K. Chang, D. Steinbach, B. Sliwa, C. Wietfeld, K. Mei, H. Shiri, H.-J. Zepernick *et al.*, "6g white paper on machine learning in wireless communication networks," *arXiv preprint arXiv:2004.13875*, 2020.

[8] D. Côté, "Using machine learning in communication networks," *Journal of Optical Communications and Networking*, vol. 10, no. 10, pp. D100–D109, 2018.

[9] I. Ahmad, S. Shahabuddin, H. Malik, E. Harjula, T. Leppänen, L. Loven, A. Anttonen, A. H. Sodhro, M. M. Alam, M. Juntti *et al.*, "Machine learning meets communication networks: Current trends and future challenges," *IEEE Access*, vol. 8, pp. 223 418–223 460, 2020.

[10] Y. Liu, F. R. Yu, X. Li, H. Ji, and V. C. Leung, "Blockchain and machine learning for communications and networking systems," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 2, pp. 1392–1431, 2020.

[11] L. U. Khan, W. Saad, Z. Han, and C. S. Hong, "Dispersed federated learning: Vision, taxonomy, and future directions," *IEEE Wireless Communications*, vol. 28, no. 5, pp. 192–198, 2021.

[12] L. Li, Y. Fan, M. Tse, and K.-Y. Lin, "A review of applications in federated learning," *Computers & Industrial Engineering*, vol. 149, p. 106854, 2020.

[13] C. Zhang, Y. Xie, H. Bai, B. Yu, W. Li, and Y. Gao, "A survey on federated learning," *Knowledge-Based Systems*, vol. 216, p. 106775, 2021.

[14] L. U. Khan, W. Saad, Z. Han, E. Hossain, and C. S. Hong, "Federated learning for internet of things: Recent advances, taxonomy, and open challenges," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 3, pp. 1759–1799, Third quarter 2021.

[15] S. Pandya, G. Srivastava, R. Jhaveri, M. R. Babu, S. Bhattacharya, P. K. R. Maddikunta, S. Mastorakis, M. J. Piran, and T. R. Gadekallu, "Federated learning for smart cities: A comprehensive survey," *Sustainable Energy Technologies and Assessments*, vol. 55, p. 102987, 2023.

[16] Y. Qu, M. P. Uddin, C. Gan, Y. Xiang, L. Gao, and J. Yearwood, "Blockchain-enabled federated learning: A survey," *ACM Computing Surveys*, vol. 55, no. 4, pp. 1–35, 2022.

[17] S. R. Pandey, N. H. Tran, M. Bennis, Y. K. Tun, A. Manzoor, and C. S. Hong, "A crowdsourcing framework for on-device federated learning," *IEEE Transactions on Wireless Communications*, vol. 19, no. 5, pp. 3241–3256, 2020.

[18] T. H. T. Le, N. H. Tran, Y. K. Tun, M. N. Nguyen, S. R. Pandey, Z. Han, and C. S. Hong, "An incentive mechanism for federated learning in wireless cellular networks: An auction approach," *IEEE Transactions on Wireless Communications*, vol. 20, no. 8, pp. 4874–4887, 2021.

[19] P. Vepakomma, O. Gupta, T. Swedish, and R. Raskar, "Split learning for health: Distributed deep learning without sharing raw patient data," *arXiv preprint arXiv:1812.00564*, 2018.

[20] C. Thapa, M. A. P. Chamikara, S. Camtepe, and L. Sun, "Splitfed: When federated learning meets split learning," *arXiv preprint arXiv:2004.12088*, 2020.

[21] V. Turina, Z. Zhang, F. Esposito, and I. Matta, "Combining split and federated architectures for efficiency and privacy in deep learning," in *Proceedings of the 16th International Conference on emerging Networking EXperiments and Technologies*, 2020, pp. 562–563.

[22] L. Liu, J. Zhang, S. Song, and K. B. Letaief, "Client-edge-cloud hierarchical federated learning," in *ICC 2020-2020 IEEE International Conference on Communications (ICC)*. IEEE, 2020, pp. 1–6.

[23] M. S. H. Abad, E. Ozfatura, D. Gunduz, and O. Ercetin, "Hierarchical federated learning across heterogeneous cellular networks," in *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 8866–8870.

[24] J. Wang, S. Wang, R.-R. Chen, and M. Ji, "Local averaging helps: Hierarchical federated learning and convergence analysis," *arXiv preprint arXiv:2010.12998*, 2020.

[25] L. U. Khan, Y. K. Tun, M. Alsenwi, M. Imran, Z. Han, and C. S. Hong, "A dispersed federated learning framework for 6g-enabled autonomous driving cars," *arXiv preprint arXiv:2105.09641*, 2021.

[26] L. U. Khan, S. R. Pandey, N. H. Tran, W. Saad, Z. Han, M. N. Nguyen, and C. S. Hong, "Federated learning for edge networks: Resource optimization and incentive mechanism," *IEEE Communications Magazine*, vol. 58, no. 10, pp. 88–93, 2020.

[27] S. Wang, T. Tuor, T. Salonidis, K. K. Leung, C. Makaya, T. He, and K. Chan, "Adaptive federated learning in resource constrained edge computing systems," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 6, pp. 1205–1221, 2019.

[28] N. H. Tran, W. Bao, A. Zomaya, and C. S. Hong, "Federated learning over wireless networks: Optimization model design and analysis," pp. 1387–1395, May 2019.

[29] G. Qu, N. Cui, H. Wu, R. Li, and Y. Ding, "Chainfl: A simulation platform for joint federated learning and blockchain in edge/cloud computing environments," *IEEE Transactions on Industrial Informatics*, vol. 18, no. 5, pp. 3572–3581, 2021.

[30] M. Chen, Z. Yang, W. Saad, C. Yin, H. V. Poor, and S. Cui, "A joint learning and communications framework for federated learning over wireless networks," *IEEE Transactions on Wireless Communications*, vol. 20, no. 1, pp. 269–283, 2020.

[31] L. U. Khan, Z. Han, D. Niyato, and C. S. Hong, "Socially-aware-clustering-enabled federated learning for edge networks," *IEEE Transactions on Network and Service Management*, vol. 18, no. 3, pp. 2641–2658, 2021.

[32] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*. PMLR, 2017, pp. 1273–1282.

[33] "Hessian matrix," https://www.algebrapracticeproblems.com/hessian-matrix/, [Online; accessed August. 6, 2022].

[34] S. Boyd, S. P. Boyd, and L. Vandenberghe, *Convex optimization*. Cambridge university press, 2004.

[35] Z. Han, M. Hong, and D. Wang, *Signal processing and networking for big data applications*. Cambridge University Press, 2017.

[36] M. Hong, X. Wang, M. Razaviyayn, and Z.-Q. Luo, "Iteration complexity analysis of block coordinate descent methods," *Mathematical Programming*, vol. 163, no. 1, pp. 85–114, 2017.
[37] S. A. Kazmi, N. H. Tran, W. Saad, Z. Han, T. M. Ho, T. Z. Oo, and C. S. Hong, "Mode selection and resource allocation in device-to-device communications: A matching game approach," *IEEE Transactions on Mobile Computing*, vol. 16, no. 11, pp. 3126–3141, 2017.
[38] 3GPP, "Evolved universal terrestrial radio access (e-utra): Physical layer procedures, release 11,," *Tech. Rep. TS 36.213*.

Latif U. Khan received his Ph.D. degree in Computer Engineering at Kyung Hee University (KHU), South Korea. Prior to that, He received his MS (Electrical Engineering) degree with distinction from University of Engineering and Technology, Peshawar, Pakistan in 2017. He is the recipient of KHU Best thesis award. He is author of two books: (a) Network Slicing for 5G and Beyond and (b) Federated Learning for Wireless Networks. He has reviewed over 200 times for the top ISI- Indexed journals and conferences. He has authored many most popular articles in the leading journals (i.e., IEEE Communications Surveys and Tutorials) and magazines (IEEE Communication Magazine, IEEE Network, and IEEE Wireless Communications Magazine). His research interests include analytical techniques of optimization and game theory to edge computing, end-to-end network slicing, wireless federated learning, and digital twins.

Mohsen Guizani (S'85, M'89, SM'99, F'09) received his B.S. (with distinction) and M.S. degrees in electrical engineering, and M.S. and Ph.D. degrees in computer engineering from Syracuse University, New York, in 1984, 1986, 1987, and 1990, respectively. He is currently a Professor with the Machine Learning Department, Mohamed Bin Zayed University of Artificial Intelligence (MBZUAI), Abu Dhabi, UAE. Previously, he served in different academic and administrative positions at the University of Idaho, Western Michigan University, the University of West Florida, the University of Missouri-Kansas City, the University of Colorado-Boulder, and Syracuse University. His research interests include wireless communications and mobile computing, computer networks, mobile cloud computing, security, and smart grid. He was the Editor-in-Chief of IEEE Network. He serves on the Editorial Boards of several international technical journals, and is the Founder and Editor-in-Chief of the Wireless Communications and Mobile Computing journal (Wiley). He is the author of nine books and more than 500 publications in refereed journals and conferences. He has guest edited a number of Special Issues in IEEE journals and magazines.

Ala Al-Fuqaha (Senior Member, IEEE) received the Ph.D. degree in computer engineering and networking from the University of Missouri-Kansas City, Kansas City, MO, USA, in 2004. He is currently a Professor at the Information and Computing Technology Division, College of Science and Engineering, Hamad Bin Khalifa University (HBKU), Doha, Qatar. His research interests include the use of machine learning in general and deep learning in particular in support of the data-driven and self-driven management of large-scale deployments of the Internet of Things (IoT) and smart city infrastructure and services, wireless vehicular networks (VANETs), cooperation and spectrum access etiquette in cognitive radio networks, and management and planning of software-defined networks (SDNs).

Choong Seon Hong (S'95-M'97-SM'11) received the B.S. and M.S. degrees in electronic engineering from Kyung Hee University, Seoul, South Korea, in 1983 and 1985, respectively, and the Ph.D. degree from Keio University, Tokyo, Japan, in 1997. In 1988, he joined KT, Gyeonggi-do, South Korea, where he was involved in broadband networks as a member of the Technical Staff. Since 1993, he has been with Keio University. He was with the Telecommunications Network Laboratory, KT, as a Senior Member of Technical Staff and as the Director of the Networking Research Team until 1999. Since 1999, he has been a Professor with the Department of Computer Science and Engineering, Kyung Hee University. His research interests include future Internet, intelligent edge computing, network management, and network security. Dr. Hong is a member of the Association for Computing Machinery (ACM), the Institute of Electronics, Information and Communication Engineers (IEICE), the Information Processing Society of Japan (IPSJ), the Korean Institute of Information Scientists and Engineers (KIISE), the Korean Institute of Communications and Information Sciences (KICS), the Korean Information Processing Society (KIPS), and the Open Standards and ICT Association (OSIA). He has served as the General Chair, the TPC Chair/Member, or an Organizing Committee Member of international conferences, such as the Network Operations and Management Symposium (NOMS), International Symposium on Integrated Network Management (IM), Asia-Pacific Network Operations and Management Symposium (APNOMS), End-to-End Monitoring Techniques and Services (E2EMON), IEEE Consumer Communications and Networking Conference (CCNC), Assurance in Distributed Systems and Networks (ADSN), International Conference on Parallel Processing (ICPP), Data Integration and Mining (DIM), World Conference on Information Security Applications (WISA), Broadband Convergence Network (BcN), Telecommunication Information Networking Architecture (TINA), International Symposium on Applications and the Internet (SAINT), and International Conference on Information Networking (ICOIN). He was an Associate Editor of the IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT and the IEEE JOURNAL OF COMMUNICATIONS AND NETWORKS. He currently serves as an Associate Editor for the International Journal of Network Management.

Dusit Niyato (M'09, SM'15, F'17) is currently a professor in the School of Computer Science and Engineering, Nanyang Technological University. He received his B.Eng. from King Mongkuts Institute of Technology Ladkrabang, Thailand, in 1999 and his Ph.D. in electrical and computer engineering from the University of Manitoba, Canada, in 2008. His research interests are in the area of energy harvesting for wireless communication, Internet of Things (IoT), and sensor networks.

Zhu Han (S'01, M'04, SM'09, F'14) received his Ph.D. degree in electrical and computer engineering from the University of Maryland, College Park. Currently, he is a professor in the Electrical and Computer Engineering Department as well as in the Computer Science Department at the University of Houston, Texas. Dr. Han is an AAAS fellow since 2019. Dr. Han is 1% highly cited researcher since 2017 according to Web of Science.